VIBE CODING CHALLENGE · FOLLOW-THROUGH

Beyond vibe coding

Building a cross-border retirement planner with Claude, without writing or reviewing a line of code

Jim Anderson · November 2026



The planner's Overview in the Cool (MSQ) look: demo build, sample data

TRY IT YOURSELF

Run the demo on a personal device

ON A LAPTOP

andersonjamesdc.github.io/calculator/download/

AS A PHONE APP

andersonjamesdc.github.io/calculator/demo/

Download the demo on a personal laptop, or scan the app code with your personal phone.

Sample data only: invented people and figures; no personal details

On a laptop: download one file and open it in any browser; every feature works, PDFs and the tax detail included, with no network

As an app: add it to the home screen; it works with no network

Stays on your device: anything you type or save stays there

AI Advisor is optional: it needs your own Anthropic API key, on the laptop and in the app

WHY THIS PROJECT

An assignment met a real problem



The assignment

Read Vibe Coding by Gene Kim and Steve Yegge, then put it into practice on something real.



The problem

Our retirement question outgrew Excel: four countries, four tax systems and their treaties, two currencies, thirty years of markets.



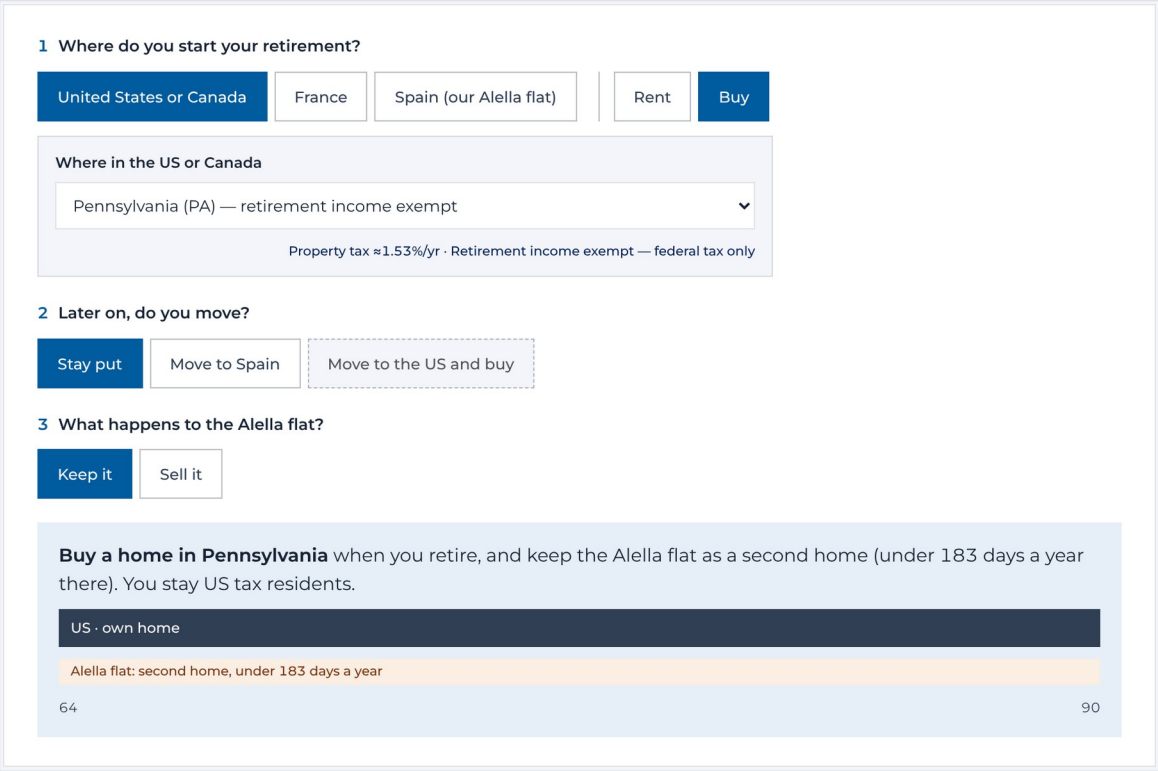
The ground rules

A personal project: off hours, my own Claude account and our own data. Nothing from work, no work systems.

The book's promise, FAAFO: fast, ambitious, autonomous, fun, optionality. My test was the "ambitious" part: could one person build something this big?

WHAT I BUILT

Our Life in Retirement: where and how we retire



Where you'll live, asked as three questions (demo build, sample data)

Six ways to retire

Spain, France, or the US or Canada; rent or buy; move later; keep or sell the Barcelona flat

Ten tax jurisdictions

39 statutory rules plus the US–Spain and US–France treaties; every figure shows its source

Risk, not just averages

Monte Carlo on 1928–2025 market history, plus currency, survivor and long-term-care stress tests

Dollars or euros

Every view in USD or EUR, at an exchange rate you set

Reports on demand

Financial plan (78 pages), Life Abroad guide (30), User Guide (29) and Tech Specs (15)

BY THE NUMBERS

One person’s spare time, six months, all off hours

260+

numbered releases, from v1 to v42.3

~7

months of off hours, from April to November 2026

31,000+

lines of application code, in 90 modules

359

automated regression checks, plus a 35-scenario golden master

~70 min

for a full test run that Claude executes unattended

0

lines of code I wrote or reviewed

HOW WE WORKED

I owned the what and the why; Claude owned the how

Me

Product owner: what to build, in what order, what "done" means

Domain expert: US, Spanish and French tax law, which I researched and kept current

Acceptance tester: push every control to extremes: is this result even possible?

Decision maker: approve major changes and settle the trade-offs

Claude

Engineer: design, code and refactoring

Tester: wrote and runs its own regression suite before every release

Researcher and explainer: plain-English reasoning and citations before any code

Release manager and writer: versions, builds, handoff notes, the User Guide and Tech Specs



HOW IT EVOLVED

The tools grew up, and so did my approach

SPRING 2026

Chat

- Claude Sonnet in a chat window
- One HTML file (3,745 lines at v10)
- I pasted console errors back by hand

SUMMER

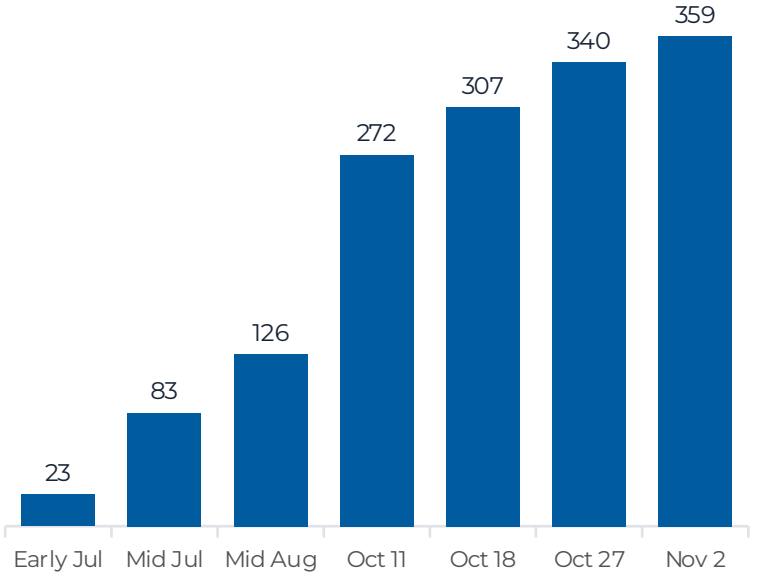
Discipline

- Long chat sessions, each ending in a handoff note
- A regression gate before every release
- Historical Monte Carlo; a full tax audit

FALL

Agentic

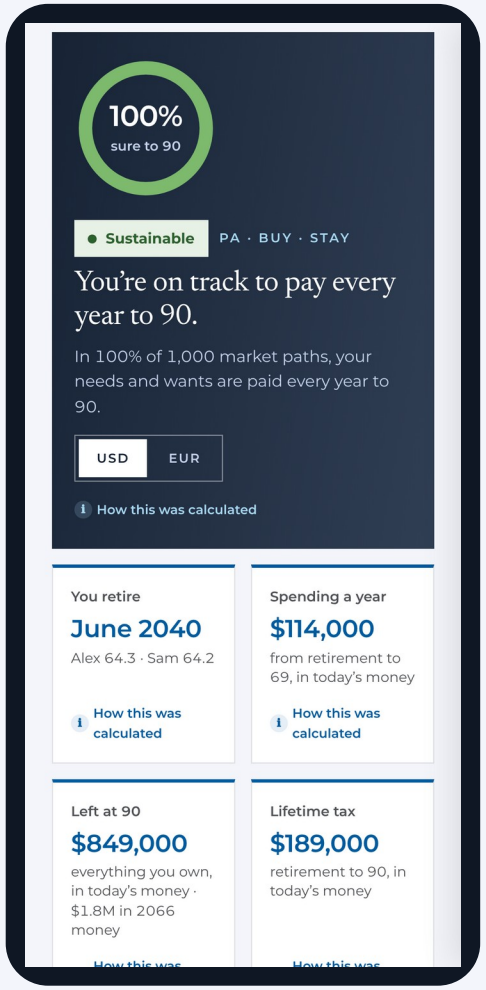
- Claude, now Opus 5.5, in its own cloud workspace
- Shell, Git, test runners, a headless browser
- Full test runs unattended; fresh agents review the work

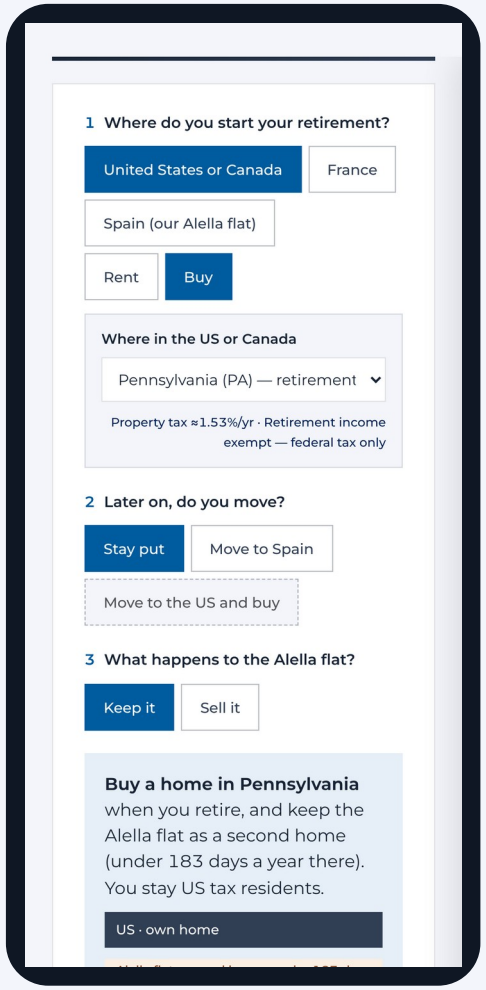


My shift: asking for features → demanding evidence → setting the engineering agenda

ON A PHONE

A plan in our pockets





Installs from a web address

Static files on GitHub Pages; add it to the home screen and it opens full screen, with no network needed

Checked on every release

Install, offline and unlock tests, and a layout audit at four phone widths in all three looks

A demo app with sample data prepopulated

Sample data at its own address, with its own icon, for anyone to try

The demo app at iPhone size, Cool (MSQ) look, sample data

WHAT IT WOULD HAVE COST

A traditional team: $0.6M–$1.9M. Mine: $200 a year

Team estimate

Two senior developers, a financial and tax analyst, a QA tester and half a project manager, for 9 to 12 months

$0.6M–$0.8M

Industry model (COCOMO II)

31,300 lines of application code: about 130 person-months over roughly 17 months, a team of seven or eight

≈ $1.9M

WHAT IT ACTUALLY TOOK

$200

a year for Claude Pro ($17 a month)

- One person
- about seven months, off hours
- 260+ releases
- every change of mind was cheap

Pay from BLS (May 2025 medians: software developers $135,980; software QA analysts and testers $104,300; financial and investment analysts, mean $112,950), times 1.4 for benefits and overhead. Excludes outside tax counsel, tools and hosting, and the value of my own time. An order of magnitude, not a quote.

SO, WAS IT VIBE CODING?

Yes, and it grew into agentic engineering

SPRING 2026 · CHAT

Vibe coding

Karpathy, Feb 2025: "When I get error messages I just copy paste them in with no comment, usually that fixes it."

For us: One HTML file. I described what I wanted and pasted console errors straight back into the chat.

SUMMER · DISCIPLINE

Head-chef vibe coding

Kim and Yegge, 2025: the developer as head chef, who delegates the cooking but owns the dish.

For us: Plain-English reasoning before code, a regression gate before every release, and my "is this even possible?" tests.

FALL · AGENTS

Agentic engineering

Karpathy, Feb 2026: "Orchestrating agents who do [the coding] and acting as oversight."

For us: Claude runs hour-long test suites and fresh-agent reviews; I set the agenda and judge the results.

I did the assignment, then followed it where the book leads.

The book's subtitle is the path I took: Building Production-Grade Software With GenAI, Chat, Agents, and Beyond. I never read the code, but I never trusted a number I hadn't tried to break.

LESSONS AND NEXT STEPS

What worked, what didn’t, what I’d suggest

What worked

- “That can’t be right” checks: nearly every one was a real bug
- Plain-English reasoning before any code
- Never ship on a failing check; every fix becomes a test
- Reviews by fresh agents that hadn’t seen the build
- A design note with a working prototype before each big feature

What didn’t

- Claude was sometimes confidently wrong on tax rules; the law stayed my job
- Memory limits: long sessions get summarized; the handoff note is past 5,800 lines
- Fragile workspaces: resets cost a test harness, then the Git history; the release files saved us
- Once, a passing result came from a stale log; Claude caught and corrected it a release later

What I’d suggest

- Train people in specification and verification, not syntax
- Guardrails first: approved accounts, data classification, human review and scanning before production